

Funções Parte II

Explicando endereços:

- &: o & do scanf("%d", &val11) por exemplo é o que permite ao scanf escrever diretamente na variável que deverá receber o valor coletado, sem trabalhar em uma "cópia".

A passagem por referência de endereços de memória resolve a questão de múltiplos retornos de uma função pois dessa forma você pode retornar mais de um valor dentro de uma função assim resolvendo o que um return sozinho não resolve. Um exemplo de endereço nativo é um vetor, um vetor já é um endereço, e sempre que um vetor é guardado em um endereço da mesma forma que uma variável ele sempre vai guardar o endereço da primeira posição e vai calcular as posições a partir da primeira.

Vetores em funções

Nas três funções lerVetor, mostrarVetor e dobrarVetor existem um detalhe muito importante: N que é passado nessas funções como parâmetro, é para a função saber seu próprio tamanho pois em C o vetor não sabe seu próprio tamanho.

Por que isso acontece em python?

Todo parâmetro em python é uma referência ao objeto, mas o comportamento dessa referência depende do seu tipo declarado, sendo os valores imutáveis aos originais, ou seja, o original que é passado na função não muda (int, float, str, bool) e os mutáveis, ou seja, o original se altera com o valor que é passado para função, pois ela não cria uma cópia do valor no endereço(list).

Os símbolos de & e * em C

variável normal guarda valor; &variável dá o endereço dela.

Variável ponteiro int *P guarda endereço; *P dá o valor que está lá.

Int A = 10

&A // endereço de A

Int *P = &A

*P //Valor de A (10)

